Búsqueda de funciones similares vía AST

Lenguajes Formales y Autómatas – UCSE – 2026.

Integrantes: Barbero Lucía, Belmonte María Luz, Galván Sol Morena

Funcionamiento

La herramienta extrae las funciones de una carpeta con archivos Python y, dada una función objetivo, devuelve las K más parecidas con su score. No compara texto sino el AST que genera ast.parse(), que es el árbol de derivación visto en la materia: el resultado de aplicar la gramática de Python a la cadena de entrada.

Decisiones de diseño

El código tiene cuatro módulos, uno por etapa: extraccion.py, normalizacion.py, similitud.py y cli.py.

Extracción. Se recorre la carpeta con os.walk, se parsea cada .py con ast.parse() y con ast.walk se toman los nodos FunctionDef y AsyncFunctionDef, incluidos métodos y funciones anidadas. De cada una se guarda nombre, archivo, línea y subárbol. Los archivos con errores de sintaxis se saltean.

Normalización. Un ast.NodeTransformer trabaja sobre una copia del subárbol. Parámetros y variables asignadas pasan a VAR1, VAR2… en orden de aparición, y un diccionario mantiene la correspondencia; los literales pasan a LIT, el nombre de la función a FUNC y se quita el docstring. Los nombres que solo se leen (print, len) se mantienen. Es la idea del analizador léxico, que entrega el token id o num y no su valor.

Serialización. Un recorrido pre-order recursivo genera la lista de tokens: primero el tipo del nodo (Assign, For, BinOp…) y luego sus hijos de izquierda a derecha. Al serializar se quita el número de las variables (VAR3 pasa a VAR), porque depende del orden de aparición: sin esto, un clon con líneas reordenadas daba 0.28 en lugar de 1.00.

Búsqueda. La función objetivo se indica como archivo.py:nombre o pegando su código. Se compara contra las n funciones del repositorio, se ordena por score y se muestran las K primeras. Cada búsqueda es O(n), dentro del $O(n^2)$ que acepta el enunciado.

Métrica de similitud

Usamos la similitud de Jaccard sobre trigramas (n = 3): el score es $|A \cap B| / |A \cup B|$, donde A y B son los conjuntos de secuencias de 3 tokens seguidos de cada función. Elegimos Jaccard y no Levenshtein porque, al trabajar con conjuntos, tolera líneas reordenadas. El valor de n y el umbral salen de comparar funciones de prueba:

Par de funciones	n = 1	n = 2	n = 3	n = 4
Clon con variables renombradas	1.00	1.00	1.00	1.00
Clon con líneas reordenadas	1.00	1.00	1.00	0.91
Funciones distintas (score más alto)	0.67	0.54	0.50	0.36

Con n = 1 casi todas las funciones comparten tipos de nodo (Assign, Return) y funciones distintas llegan a 0.67; con n grande, cualquier cambio pequeño rompe los n-gramas. Con n = 3 los clones dan 1.00 y las funciones distintas, como máximo, 0.50. Por eso fijamos el umbral de duplicado en 0.7: con un score mayor o igual, dos funciones son candidatas a duplicado.

Tests

La suite (test.py, con pytest) tiene dos positivos, un clon con variables renombradas y uno con líneas reordenadas (1.00 ambos), y dos negativos: funciones totalmente distintas (0.13) y sumar una lista contra buscar su máximo (0.50). Los cuatro pasan.

Limitaciones conocidas

La herramienta compara estructura, no comportamiento. Por eso no detecta:

- Clones semánticos: sumar una lista con un for contra sum(lista).
- Cambios de estructura de control: un for reemplazado por un while.
- Clones con muchas líneas agregadas o eliminadas.
- Clones que usan variables globales con distinto nombre, que no se normalizan.

Además, Jaccard cuenta una sola vez cada trigrama repetido, y un único LIT no distingue números de strings.

Posibles mejoras

- Reemplazar los literales por su tipo (NUM, STR) en lugar de un único LIT.
- Contar los n-gramas con su frecuencia, o combinar Jaccard con Levenshtein.
- Normalizar estructuras equivalentes, como for y while, antes de comparar.